Sensitive Information Types Implementation — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Sensitive Information Types Implementation (Microsoft Purview) 4

The problem . . . 4

The solution . . . 4

What it delivers . . . 4

Business outcomes . . . 4

Boundaries (so scope is clear) . . . 5

1. Solution Overview — Sensitive Information Types Implementation (Microsoft Purview) 5

What this solution is . . . 5

Why this matters . . . 6

How it works . . . 6

Confidence levels & instance count (the two tuning dials) . . . 7

Where selected SITs are consumed (why accuracy compounds) . . . 7

Naming convention (used in every doc and script) . . . 7

Limitations (be honest with buyers) . . . 8

The standard deployment model . . . 8

2. Requirements — Sensitive Information Types Implementation (Microsoft Purview) 8

2.1 Licensing sufficient to view, test, and use SITs (and to see where they match) . . . 8

2.2 SIT-specific least-privilege roles — including Content Explorer access . . . 9

2.3 A classification intent and a data-category inventory . . . 9

2.4 A representative test corpus of clearly-fake sample data . . . 10

2.5 DLP available for simulation-based measurement . . . 10

2.6 A false-positive threshold and a pilot scope (SIT-specific) . . . 10

Global Requirements — Microsoft Purview Solutions 11

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 12

2. Purview roles (least-privilege) and portal access . . . 12

3. Security & Compliance PowerShell + Graph toolchain . . . 12

4. Data sources & scope in scope . . . 13

5. Compliance, records, privacy & legal stakeholders . . . 13

6. A pilot scope and change control . . . 13

3. Implementation Guide — Sensitive Information Types Implementation (Microsoft Purview) 14

Phase 0 — Connect and baseline (Assess) . . . 14

Phase 1 — Select the built-in SITs (Design → `SIT Selection Register`) . . . 14

Phase 2 — Test each selected SIT against the corpus (Configure → prove accuracy) . . . 15

Phase 3 — Measure against real traffic in SIMULATION (Configure — no user impact) . . . 16

Phase 4 — Tune the SIT usage (drive false positives down) . . . 16

Phase 5 — Validate and hand off to downstream controls (Expand) . . . 17
Order-of-operations summary . . . 17
5. Testing & Validation — Sensitive Information Types Implementation (Microsoft Purview) 18
Test A — True-positive detection (the SIT finds what it should) . . . 18
Test B — Negative rejection & false-positive rate (the adoption gate) . . . 19
Test C — Consistency & hand-off (the selection is usable downstream) . . . 20
Safety & rollback validation . . . 20
Evidence to capture (for the deliverable record) . . . 21
6. Troubleshooting — Sensitive Information Types Implementation (Microsoft Purview) 21
Connectivity & cmdlet availability . . . 21
Discovery & selection (Phase 0–1) . . . 22
Testing against the corpus (Phase 2) . . . 22
Building the simulation policy (Phase 3) . . . 22
Scoping to a pilot ring (CLSSIT-Set-Scope.ps1) . . . 22
Measuring & tuning (Phase 4) . . . 23
Validation & handoff (Phase 5) . . . 23
What this solution does NOT decide . . . 23
7. Best Practices — Sensitive Information Types Implementation (Microsoft Purview) 23
Reuse built-in before you build anything . . . 23
Tune the two dials before touching the pattern . . . 24
Test with fake data, both ways . . . 24
Measure against reality, in simulation . . . 24
Keep the selection consistent and documented . . . 25
Stay inside the boundary . . . 25
8. Deliverables Summary — Sensitive Information Types Implementation (Microsoft Purview) 25
Documentation . . . 25
PowerShell tools (5) — Security & Compliance PowerShell / Graph . . . 26
Definition of done . . . 27
Appendix A — PowerShell Scripts 29

Executive Summary — Sensitive Information Types Implementation (Microsoft Purview)

The problem

You can't protect what you can't detect. Every data-protection control in Microsoft Purview — DLP, auto-labeling, retention, insider risk — depends on the same underlying layer: **sensitive information types (SITs)**, the classifiers that recognize credit-card numbers, national IDs, health identifiers, and hundreds of other data patterns. If that detection layer is inaccurate — missing real sensitive data, or firing on internal reference numbers that just *look* sensitive — every control built on top of it is either blind or drowning in false positives. The common failure is switching on a built-in detector at its default settings, without ever testing it, and watching false positives train users to ignore every alert.

The solution

Select, test, and tune the built-in SITs Microsoft already ships — a large, tested library covering the common data patterns — using only the Microsoft Purview portal, Security & Compliance PowerShell, and Microsoft Graph. Each in-scope data category is mapped to the most specific built-in SIT, **tested against a corpus of clearly-fake positive and look-alike-negative samples**, then **measured against real production content in a DLP policy run in SIMULATION** (zero user impact) and **tuned** using the two dials that matter — **confidence level** and **instance count** — until the false-positive rate is at/under an agreed threshold. Where the shipped SIT genuinely can't separate real data from look-alikes, an **editable copy** is tuned — the original built-in is never touched. The result is a detection layer the whole Purview estate can trust.

What it delivers

- A **category→SIT selection register** — the right built-in detector for each in-scope data category, reusing built-in wherever it fits and flagging only genuine gaps for the custom/EDM/trainable solutions.
- **Tested accuracy** — every selected SIT proven to detect true positives and reject look-alike negatives against a fake-data corpus, in the portal and at scale.
- A **measured, tuned false-positive rate** kept at/under an agreed threshold — the difference between a detector people trust and one they ignore — with matches inspected in Content Explorer.
- A **threshold register** (confidence + instance count per SIT) that every downstream control inherits, so DLP, auto-labeling, and retention all detect consistently.
- **Five PowerShell tools** (SIT inventory, accuracy tester, simulation measurement builder, match/false-positive report, scope helper) — read-only or `-WhatIf` by default, simulation-only, no credentials in code.

Business outcomes

- **Accurate detection** of the data that matters, across every control that consumes it — because the

detection layer was tested, not assumed.

- **Low false positives** — simulation-first measurement plus dial-tuning means the detector catches real sensitive data without a wave of noise that gets it disabled.
- **No user disruption** — this solution introduces **no enforcement**; all measurement is in simulation or the portal Test experience.
- **Self-guided at a fraction of consulting cost** — the same method a specialist would use, documented and tooled to run in-house.

Boundaries (so scope is clear)

This solution covers **built-in** SITs; custom SITs, Exact Data Match, and trainable classifiers for gaps are adjacent solutions. Several measurement surfaces (Content/Activity Explorer, auto-labeling) are **licensing-/plan-dependent** (E5 / E5 Compliance; verify your tenant). The solution provides the **technical selection, testing, and tuning method and the evidence**; *which* data categories are in scope, *which* SITs legally satisfy a regulation, and the **acceptable false-positive threshold** are **compliance, privacy, and legal determinations**, captured alongside the delivered evidence.

1. Solution Overview — Sensitive Information Types Implementation (Microsoft Purview)

What this solution is

Every data-protection control in Microsoft Purview — DLP, auto-labeling, retention, insider risk, DSPM, Communication Compliance — keys off the same underlying question: *does this content contain sensitive data, and how confident are we?* The answer comes from **sensitive information types (SITs)** — the pattern-based classifiers Purview uses to recognize credit-card numbers, national IDs, health identifiers, bank account numbers, and hundreds of other data patterns. Microsoft ships a large, growing library of **built-in SITs** (verify the current count in your tenant), each with a tested regex/checksum pattern, **supporting keyword/format evidence**, and defined **confidence levels**.

This solution is a repeatable, evidence-driven way to **select the right built-in SITs for your in-scope data categories, test them against representative sample content, and tune how they are applied** — confidence level and instance count — so the detection layer is accurate *before* it drives any policy. It uses only the **Microsoft Purview portal**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** — no third-party classification tooling. Its defining discipline is that SIT accuracy is **measured** (in the portal Test experience and in a DLP policy run in **SIMULATION mode**) and **tuned for a low false-positive rate**, not assumed.

> ***Scope boundary.*** *This solution delivers the **technical method and evidence** for selecting, testing, and tuning **built-in** SITs. *Which* data categories are in scope, *which* SITs legally satisfy a regulation, and the *acceptable* false-positive threshold are **business, compliance, privacy, and legal determinations** — deferred to those functions. Building **custom** SITs, **Exact Data Match**, and **trainable classifiers** for gaps built-in SITs can't cover are adjacent solutions; here we get the built-in*

library right first.

Why this matters

• **Built-in SITs are the fastest path to coverage** — a large tested library exists out of the box, so

most common data categories (payment cards, SSNs/national IDs, IBANs, health record numbers) are already covered without authoring anything. Selecting well beats building custom.

• **The default confidence and instance count are where accuracy is won or lost.** A built-in *Credit Card

Number* SIT firing at a single low-confidence match on internal reference numbers trains users to ignore every alert. Choosing **High confidence** and a realistic **instance count** is the single biggest false-positive lever — and it costs nothing to change.

• **You cannot edit a built-in SIT's pattern — but you can test it and tune its use, and copy it to tune

further. **Purview lets you** create an editable copy** of a built-in SIT when the shipped one over/under-matches your data; knowing when to reach for that (versus just raising confidence/instance count) is the skill this solution teaches.

• **The same SIT selection drives the whole Purview estate.** Get the built-in SIT layer accurate here and

DLP, auto-labeling, retention, and insider risk all inherit trustworthy detection.

How it works

1. **The built-in SIT library defines what Purview can detect.** Each built-in SIT is a tested pattern

(regex + checksum where applicable) plus **supporting evidence** (keywords, proximity, format) and **confidence levels** — **Low / Medium / High** (surfaced numerically as roughly **65 / 75 / 85**). A higher confidence level means more corroborating evidence was found, so fewer false positives.

1. **Selection** maps each in-scope data category to the built-in SIT (or small set of SITs) that detects

it — reusing built-in wherever it fits, flagging only genuine gaps for the custom/EDM/trainable solutions.

1. **Testing** proves each selected SIT actually detects your representative sample content and does **not**

fire on look-alike non-sensitive content — using the portal **Test** experience (upload a file, see matches + confidence) and, at scale, a **DLP policy run in SIMULATION** against real traffic.

1. **Tuning** adjusts *how the SIT is used* — the **confidence level** and **instance count (min/max)** it

triggers at — measured against the test/simulation results, to drive the false-positive rate down while keeping true positives. Where the shipped SIT still over/under-matches, an **editable copy** of the built-in SIT is tuned (supporting keywords, proximity, confidence).

1. **Validation** confirms, against explicit pass/fail gates, that each category's SIT detects the true

positives, rejects the negatives, and sits at/under the agreed false-positive rate.

1. **Adoption** hands the tuned, validated SIT selection to the downstream controls (DLP, auto-labeling,

retention) with a documented **threshold register** so every policy uses the same settings.

Confidence levels & instance count (the two tuning dials)

• **Confidence level (Low/Medium/High).** How much corroborating evidence Purview found. **Start at High**

for most categories — it requires the strongest evidence (correct format *plus* checksum *plus* nearby keywords) and dramatically cuts false positives. Lower only when High misses true positives you can prove are real.

• **Instance count (min / max).** How many distinct matches must appear in one item to trigger. A single

card number is noise; **ten in one file** is a likely bulk leak. Raising the minimum instance count focuses detection on exfiltration-shaped events and cuts single-hit false positives.

• These two dials are set **per use** (per DLP rule / auto-label rule) — the built-in SIT is shared, but

each policy chooses the confidence and instance count. This solution records them once in a **threshold register** so every downstream policy is consistent.

Where selected SITs are consumed (why accuracy compounds)

• **Data classification** → **Content explorer / Activity explorer** — see *where* SIT-matched content lives and

what is happening to it (requires the Content Explorer roles).

• **Data Loss Prevention** — SITs as rule conditions across email, cloud, Teams, endpoint.

• **Auto-labeling** (sensitivity and retention) — apply labels automatically where a SIT matches.

• **Insider Risk, Communication Compliance, DSPM** — SIT signals feed risk and posture.

One accurate SIT selection here is inherited by all of the above — which is why testing and tuning the built-in layer first pays off across the estate.

Naming convention (used in every doc and script)

Every artifact this solution creates uses a consistent name, reused verbatim across the docs, scripts, and the Purview portal. The standard set:

Artifact	Name
Category → SIT map (register)	`SIT Selection Register`
Confidence/instance settings record	`SIT Threshold Register`
Test corpus (fake sample data)	`SIT Test Corpus – <Category>`
Measurement DLP policy (simulation)	`SIT Measure – <Category> – Simulation`
Editable copy of a built-in SIT (only if needed)	`<Built-in name> – Tuned Copy – Classification`
Pilot user ring	`SIT Pilot Ring – Stage1`

Limitations (be honest with buyers)

• **You cannot change a built-in SIT's shipped pattern.** You tune its *use* (confidence, instance count)

or create an **editable copy** to adjust supporting evidence — but the original built-in pattern is fixed.

• **Some SITs and testing features are licensing-/plan-dependent or in preview**, and the portal evolves —

this solution flags **"verify your licensing/tenant"** rather than asserting availability or an exact library count that may differ in your tenant.

• **A SIT detects a *pattern*, not *meaning*.** A built-in number pattern will match any content in that

format — which is exactly why testing and tuning against your real look-alike data matters. Categories that are conceptual rather than pattern-based (e.g. source code, contracts) need **trainable classifiers**, not a built-in SIT.

• **Which SITs legally satisfy a regulation, and the acceptable false-positive threshold, are

compliance/legal determinations** — this solution configures and proves the detector; it does not make the determination.

The standard deployment model

> ***Assess*** *the built-in SIT library against your in-scope data categories →* ***Design*** *the category→SIT selection and target confidence/instance settings →* ***Configure*** *the test corpus and a* ***simulation*** *measurement policy →* ***Pilot*** *(test → measure → tune) →* ***Validate*** *against pass/fail gates (true positives detected, negatives rejected, false-positive rate ≤ threshold) →* ***Expand*** *by handing the validated selection + threshold register to the downstream controls. Every measurement is done in the portal Test experience or in* ***simulation*** *first — no enforcement is introduced by this solution.*

2. Requirements — Sensitive Information Types Implementation (Microsoft Purview)

These are the **solution-specific** prerequisites for selecting, testing, and tuning **built-in** sensitive information types (SITs) in Microsoft Purview. They build on — and do not repeat — the six shared prerequisites in `global-requirements-purview.md` (tenant + compliance licensing, least-privilege Purview roles, S&C PowerShell + Graph toolchain, in-scope data sources, compliance/records/privacy/legal stakeholders, and a pilot scope + change control). Verify each item below before starting the Implementation Guide.

For every requirement: **Description** (what it is), **Why required**, **How to verify**, **How to implement if missing**.

2.1 Licensing sufficient to view, test, and use SITs (and to see where they match)

• **Description:** The built-in SIT **library** and basic SIT use in DLP are broadly available, but the

surfaces you tune and validate against — **Content explorer**, **Activity explorer**, **auto-labeling**, and advanced classifiers — are **plan-gated** (typically **Microsoft 365 E5** or the **E5 Compliance** add-on; some capabilities are in **preview**). This extends GR-#1 with the SIT-specific detail.

• **Why required:** You can *select* a built-in SIT on most plans, but *measuring* its false-positive rate

in Content/Activity Explorer — the core of tuning — depends on the compliance plan. Testing a SIT against real-traffic volume via a simulation DLP policy needs DLP available.

• **How to verify:** In the **Microsoft 365 admin center** confirm the subscription/add-ons; in the

Purview portal → **Data classification** confirm **Content explorer** and **Activity explorer** are visible, and **Data loss prevention** is available for simulation measurement. **Flag "verify your licensing/tenant"** — availability evolves.

• **How to implement if missing:** Assign/acquire the required licensing (E5 / E5 Compliance) before

relying on Content/Activity Explorer for tuning; on lower plans, use the portal **Test** experience and a simulation DLP policy for measurement instead.

2.2 SIT-specific least-privilege roles — including Content Explorer access

• **Description:** The Purview roles needed to work with SITs: **Compliance Administrator** or **Compliance

Data Administrator **to view the SIT library and create/tune the measurement policy and any editable SIT copy;** DLP Compliance Management **for the simulation policy; and — critically — the** Content Explorer List viewer **and** Content Explorer Content viewer** roles to *see the actual matched items* when tuning false positives. Global Administrator only where genuinely required. Extends GR-#2.

• **Why required:** Tuning is impossible without seeing what matched. The Content Explorer roles are a

separate, deliberately restricted grant (they expose sensitive content) — without them you can count matches but not inspect them to classify true vs false positive.

• **How to verify: Purview portal** → **Settings** → **Roles & scopes** → **Permissions** — confirm your account's

role-group membership includes the compliance and **Content Explorer** roles.

• **How to implement if missing:** Assign the least-privilege role groups under change control; treat the

Content Explorer content-viewer grant as sensitive and time-boxed to the tuning exercise.

2.3 A classification intent and a data-category inventory

• **Description:** A documented view of **what sensitive data you are protecting** (e.g. payment card data,

national IDs, health identifiers, bank account numbers, customer PII) so each category can be mapped to a built-in SIT. This is the input to the `SIT Selection Register`. Extends GR-#4.

• **Why required:** Selection is category-driven: without knowing which data categories matter, you cannot

choose the right built-in SITs or judge whether a match is a true or false positive.

• **How to verify:** Confirm the in-scope data categories with the data owners; **Purview portal → Data classification → Classifiers → Sensitive info types** lists the built-in SITs available to map them to.

• **How to implement if missing:** Build the category inventory with stakeholders before selecting SITs (this solution's Discovery script inventories the built-in library to map against).

2.4 A representative test corpus of clearly-fake sample data

• **Description:** A small **test corpus** of **non-real, clearly-fake** sample content per category — both **positive** samples (content that *should* match, using documented fake test values such as the test credit-card number `4111 1111 1111 1111`) and **negative / look-alike** samples (internal reference numbers, order IDs, and other content that *resembles* the pattern but must **not** match). This becomes the `SIT Test Corpus – <Category>`.

• **Why required:** Testing and tuning require known inputs. Without a corpus you cannot prove a SIT detects true positives *and* rejects the look-alikes that drive false positives — which is the whole point.

• **How to verify:** Confirm you have positive and negative samples per category and that **no real customer/employee data** is used.

• **How to implement if missing:** Assemble the corpus from documented test values and synthetic look-alike content before testing. **Never** use production sensitive data in a test corpus.

2.5 DLP available for simulation-based measurement

• **Description:** For measuring a SIT's false-positive rate against **real traffic** (not just a static corpus), a **DLP policy run in SIMULATION mode** referencing the selected SITs — the `SIT Measure – <Category> – Simulation` policy. Simulation evaluates against live content and records what *would* match, with **zero** user impact.

• **Why required:** The portal Test experience proves a SIT against your corpus; simulation proves it against the messy reality of production content, which is where the real false-positive drivers surface. Simulation is the safe way to do this without touching a single user.

• **How to verify: Purview portal** → **Data loss prevention** → **Policies** — confirm you can create a policy and select **Run the policy in simulation mode**.

• **How to implement if missing:** If DLP is not in your plan, tune against the portal Test experience and Content Explorer only; acquire DLP for real-traffic measurement where the category warrants it.

2.6 A false-positive threshold and a pilot scope (SIT-specific)

• **Description:** An agreed **false-positive threshold** per category (the maximum acceptable share of

matches that are false positives — a compliance/privacy decision), an agreed **simulation dwell time** (long enough to capture representative traffic), and a named `SIT Pilot Ring – Stage1` to measure against before the selection is adopted tenant-wide. Extends GR-#6.

• **Why required:** "Tuned" is meaningless without a target. The threshold defines *done*; the dwell time ensures the measurement is representative; the ring keeps measurement scoped.

• **How to verify:** Confirm the threshold, dwell time, and ring membership with compliance and the data owners.

• **How to implement if missing:** Agree the threshold and dwell time and define the ring before measuring;

record them in the `SIT Threshold Register`.

Requirements summary

#	Requirement	Gate
GR 1–6	Category prerequisites (`global-requirements-purview.md`)	Tenant/licensing, roles, toolchain, scope, stakeholders, pilot + change control
2.1	Licensing to view/test/measure SITs (E5 / E5 Compliance; verify)	Content/Activity Explorer + DLP simulation available
2.2	SIT roles **incl. Content Explorer viewer**	Can see & classify matched items
2.3	Classification intent + data-category inventory	Selection is category-driven
2.4	Fake positive + negative test corpus	Testing has known inputs
2.5	DLP available for simulation measurement	Real-traffic false-positive measurement
2.6	FP threshold + dwell time + pilot ring	"Tuned" has a defined target

> ***Deferred to compliance/privacy/legal:*** *which data categories are in scope, which SITs legally satisfy a regulation, and the acceptable false-positive threshold per category. This solution provides the technical selection, test, and tuning method and the evidence — not the determination.*

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet*

*detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

• **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

• **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

• **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

• **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

• **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

• **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.

• **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.

• **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

• **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

• **Why required:** Configuration, automation, and the provided scripts run through these.

• **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.

• **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

• **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams, endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

• **Why required:** Purview policies key off the in-scope workloads and the classification scheme.

• **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with stakeholders.

• **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

• **Description:** Engagement with the **compliance, records-management, privacy, security, and legal** stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

• **Why required:** Purview configuration must align to enterprise policy and legal/records requirements, not technical defaults; several capabilities have HR/legal sensitivity.

• **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.

• **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

• **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

• **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate → Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

• **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change window/approval path.

• **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Sensitive Information Types Implementation (Microsoft Purview)

This is the exact, ordered procedure to **select, test, and tune built-in sensitive information types (SITs)** so the detection layer is accurate before it drives any policy. All measurement is done in the portal **Test** experience or in a DLP policy run in **SIMULATION** — this solution introduces **no enforcement**. Portal paths and cmdlets are from Microsoft's public documentation; **licensing-/plan- dependent features and evolving portal paths are flagged "verify your licensing/tenant"** and are never asserted for your tenant. Do the phases in order. Each phase names its artifacts (from the Overview naming table), states a **pass/fail gate**, and gives a **rollback**.

> ***Automation.*** *The five delivered PowerShell tools run through **Security & Compliance PowerShell** (`Connect-IPPSSession`) and **Microsoft Graph**. They are **read-only or** `-WhatIf` **by default**; the one tool that writes (the simulation measurement policy) requires an explicit `-Commit` and creates the policy **in simulation mode only — never enforce**. No credentials are stored in code.*
>
> ***Determinations deferred.*** **Which* data categories are in scope, *which* SITs legally satisfy a regulation, and the *acceptable* false-positive threshold are **compliance/privacy/legal** decisions. Capture them before Phase 3.*

Phase 0 — Connect and baseline (Assess)

1. **Connect the toolchain** (GR-#3). On the admin workstation:

```
Install-Module ExchangeOnlineManagement -Scope CurrentUser -Force   # provides Connect-IPPSSession
Connect-IPPSSession -UserPrincipalName admin@contoso.com            # Security & Compliance PowerShell
Connect-MgGraph -Scopes 'InformationProtectionPolicy.Read','Organization.Read.All' -NoWelcome
```

1. **Inventory the built-in SIT library** (delivered script #1, read-only):

```
.\CLSSIT-Discovery.ps1 -CsvPath .\CLSSIT-SIT-Inventory.csv
```

It lists the built-in SITs available in your tenant, the **confidence levels** each publishes, and cross-references which existing DLP/auto-labeling policies already reference SITs — so you build on what exists.

1. **Confirm licensing and roles** (Req 2.1, 2.2): confirm the compliance plan and that you hold the

Content Explorer viewer roles needed to inspect matched items during tuning.

Pass/fail gate: you have a written inventory of the built-in SITs available in your tenant, their confidence levels, and confirmed Content Explorer access for tuning. **Rollback:** none (read-only).

Phase 1 — Select the built-in SITs (Design → `SIT Selection Register`)

Map each in-scope data category to the built-in SIT (or minimal set) that detects it. Reuse built-in wherever it fits; flag only genuine gaps for the custom/EDM/trainable solutions.

1. **List the built-in candidates per category.** In **Purview portal → Data classification → Classifiers →

Sensitive info types**, find the built-in SITs for each category — e.g. *Credit Card Number*, *U.S. Social Security Number (SSN)*, *ABA Routing Number*, *International Banking Account Number (IBAN)*, the relevant national-ID and health-identifier SITs. Read each SIT's definition (pattern elements, supporting keywords, and its** Low/Medium/High confidence levels**).

1. **Choose one primary SIT per category** (occasionally a small set) and record it in the

`SIT Selection Register`: category → chosen built-in SIT → why → the SITs considered and rejected. Prefer the most specific built-in SIT (a country-specific national-ID SIT over a generic number pattern).

1. **Record the target settings** for each in the `SIT Threshold Register`: the starting **confidence

level **(default** High**) and** instance count (min/max)** you will test at. These are the tuning dials, set per downstream use.

1. **Flag genuine gaps.** Any category with no fitting built-in SIT (an internal Employee ID; a conceptual

category like source code) is **out of scope here** — note it for the custom-SIT / EDM / trainable- classifier solutions. Do **not** invent a built-in SIT that doesn't exist.

Pass/fail gate: every in-scope data category maps to a named built-in SIT with a starting confidence and instance count recorded, or is explicitly flagged as a gap for another solution. **Rollback:** none (documentation only — no tenant change yet).

Phase 2 — Test each selected SIT against the corpus (Configure → prove accuracy)

Prove each SIT detects your **positive** samples and rejects your **negative / look-alike** samples, using clearly-fake data (Req 2.4).

1. **Portal Test experience.** For each selected SIT: **Data classification → Classifiers → Sensitive info

types → [SIT] → Test**, then upload a sample file from the** `SIT Test Corpus – <Category>`**. The portal reports whether the SIT matched and at** what confidence **— confirm positives match (ideally at High) and look-alike negatives do** not**.

1. **Test at scale from PowerShell** (delivered script #2, read-only):

```
.\CLSSIT-Test.ps1 -SensitiveType 'Credit Card Number' -Path .\corpus\payment-card\ -CsvPath .\CLSSIT-Te
```

It runs each corpus file through the SIT and reports match/no-match and confidence per file — so you can score the whole corpus in one pass. (Uses `Test-DataClassification` where available in your tenant; verify the cmdlet is present — the script degrades to guidance if not.)

1. **Score the corpus:** for each SIT, confirm **positives detected** (true-positive coverage) and

negatives rejected (no look-alike matches) at the target confidence. Note any positive that only matches below High, or any negative that matches — these drive Phase 3 tuning.

Pass/fail gate: each selected SIT detects its positive corpus at the target confidence and rejects its negative corpus, or the misses/false hits are captured for tuning. **Rollback:** none (read-only testing).

Phase 3 — Measure against real traffic in SIMULATION (Configure — no user impact)

The corpus proves the SIT in the lab; simulation proves it against production content, where the real false-positive drivers appear. Capture the false-positive threshold (Req 2.6) before this phase.

1. **Create the measurement policy in simulation** (delivered script #3, `-WhatIf` by default):

```
# Preview:
.\CLSSIT-BuildSimPolicy.ps1 -PolicyName 'SIT Measure - Payment Card - Simulation' `
    -SensitiveTypes 'Credit Card Number' -MinCount 10 -Confidence High -WhatIf
# Create it (SIMULATION mode only — never enforce):
.\CLSSIT-BuildSimPolicy.ps1 -PolicyName 'SIT Measure - Payment Card - Simulation' `
    -SensitiveTypes 'Credit Card Number' -MinCount 10 -Confidence High -Commit
```

Portal equivalent: **Data loss prevention** → **Policies** → **Create policy** → custom → **all/needed locations** → rule condition *Content contains* the selected SIT at the chosen confidence/instance count → **Policy mode: Run the policy in simulation mode**.

1. **Scope the measurement** to `SIT Pilot Ring – Stage1` if you want to limit the sample (delivered script

#5, `-WhatIf` by default):

```
.\CLSSIT-Set-Scope.ps1 -PolicyName 'SIT Measure - Payment Card - Simulation' -IncludeGroup 'SIT Pilot
```

1. **Let it dwell** for the agreed dwell time (Req 2.6) to capture representative traffic. Simulation

records what *would* have matched with **no** enforcement and **no** user impact.

Pass/fail gate: the measurement policy exists and is confirmed in **simulation** (script #1/#3 report a `Test*` mode), scoped correctly, capturing matches. **Rollback:** delete the simulation policy — simulation has no user impact, so removal is clean.

Phase 4 — Tune the SIT usage (drive false positives down)

1. **Export and classify the matches** (delivered script #4, read-only):

```
.\CLSSIT-Report.ps1 -Days 14 -CsvPath .\CLSSIT-Matches.csv
```

Cross-reference with **Data classification** → **Content explorer** (inspect the actual matched items — needs the Content Explorer viewer role, Req 2.2) and **Activity explorer**. Classify each match **true positive** or **false positive** and identify the driver.

1. **Tune the two dials first** (no SIT copy needed):

• **Raise the confidence level** (e.g. Medium → High) so only strongly-corroborated matches count.

• **Raise the instance count** (e.g. require ≥10 matches, not 1) to focus on bulk-leak-shaped events and

suppress single-hit false positives. Update the `SIT Threshold Register` and re-measure.

1. **Only if the two dials aren't enough, create an editable copy of the built-in SIT.** Portal:

Sensitive info types → **[built-in SIT]** → **Copy**, producing `<Built-in name> – Tuned Copy – Classification`; then adjust its **supporting keywords, character proximity, or confidence** to fit your data (e.g. require a nearby keyword your internal look-alikes lack). Test the copy against the corpus (Phase 2) and point the measurement policy at the copy. *The original built-in SIT is never modified — only the copy.*

1. **Re-measure after each change** and repeat until the false-positive rate meets the threshold. Change one

thing at a time — don't stack changes blind.

Pass/fail gate: measured false-positive rate is at/under the agreed threshold on the simulation sample, with true positives still detected. **Rollback:** revert the threshold change in the register / delete the tuned copy — the built-in SIT and any downstream policy are unaffected.

Phase 5 — Validate and hand off to downstream controls (Expand)

1. **Run the full validation** (next section) and record the pass/fail evidence per category: positives

detected, negatives rejected, false-positive rate ≤ threshold.

1. **Finalize the registers.** Lock the `SIT Selection Register` (category → SIT) and **`SIT Threshold

Register`** (confidence + instance count per SIT) as the authoritative settings.

1. **Hand off to the downstream controls.** DLP, auto-labeling, and retention policies now reference the

validated selection at the **registered** confidence/instance count — so every control inherits the same tuned detection. (Standing up those enforcement policies is the DLP / labeling / retention solutions' job; this solution delivers the accurate SIT layer they consume.)

1. **Decommission the measurement policy** once the selection is validated — delete the

`SIT Measure – <Category> – Simulation` policies (they were measurement scaffolding, never enforcement). Schedule a periodic re-validation as data and the built-in library evolve.

Pass/fail gate: every in-scope category has a validated SIT at a registered confidence/instance count, evidence captured, and the registers handed to the downstream controls. **Rollback:** the registers are documentation; downstream policies own their own enforcement rollback. This solution leaves no enforcement behind.

Order-of-operations summary

Phase	Do	Artifact(s)	Gate	Rollback
0 Assess	Connect, inventory built-in SITs, confirm roles	SIT inventory CSV	Inventory + Content Explorer access	n/a (read-only)
1 Select	Map category → built-in SIT, set target dials	Selection + Threshold registers	Every category → named SIT or flagged gap	n/a (docs only)
2 Test	Prove SIT vs fake corpus (portal + script)	Test corpus, results CSV	Positives detected, negatives rejected	n/a (read-only)
3 Measure	Build **simulation** policy, dwell	`SIT Measure – … – Simulation`	In simulation, capturing matches	Delete sim policy

Phase	Do	Artifact(s)	Gate	Rollback
4 Tune	Raise confidence/instance; copy SIT if needed	Threshold register / tuned copy	FP rate ≤ threshold	Revert register / delete copy
5 Validate	Validate, finalize registers, hand off	Locked registers + evidence	Validated selection handed off	Downstream owns enforcement

5. Testing & Validation — Sensitive Information Types Implementation (Microsoft Purview)

Validate the built-in SIT selection against explicit **pass/fail gates**. The three headline gates are: **(A) true positives are detected**, **(B) look-alike negatives are rejected and the false-positive rate is low**, and **(C) the selection is consistent and handed off correctly**. Run every test with **non-real, clearly-fake sample data** (e.g. the Microsoft-documented test credit-card number `4111 1111 1111 1111`) — never real customer/employee data — and record the evidence.

> *All tests use the* `SIT Test Corpus – <Category>` *(Req 2.4) and, for real-traffic measurement, the* `SIT Measure – <Category> – Simulation` *policy — always in* ***simulation****, never enforced. This solution introduces no enforcement.*

Test A — True-positive detection (the SIT finds what it should)

#	Test	Steps	Pass condition
A1	Portal Test — positive	In **Sensitive info types** → **[SIT]** → **Test**, upload a **positive** corpus file (contains valid fake instances).	SIT matches at the **target confidence** (ideally High).
A2	Scripted corpus — positives	Run `CLSSIT-Test.ps1 -SensitiveType '<SIT>' -Path .\corpus\<category>\positives\`.	Every positive sample is matched; confidence recorded per file.
A3	Instance-count boundary	Provide a file with **exactly** the minimum instance count, and one **just below** it.	At-threshold file matches; below-threshold file correctly does **not** — confirms the instance dial.

#	Test	Steps	Pass condition
A4	Confidence boundary	Provide a positive with strong corroborating evidence and one with weak evidence.	Strong-evidence sample matches at High; weak-evidence sample matches only at lower confidence (or not) — confirms the confidence dial behaves as expected.

Gate A (pass): every category's positive corpus is detected by its selected SIT at the target confidence, and the instance/confidence boundaries behave as configured.

Test B — Negative rejection & false-positive rate (the adoption gate)

#	Test	Steps	Pass condition
B1	Portal Test — look-alike negative	Upload a **negative / look-alike** corpus file (internal reference numbers, order IDs that resemble the pattern).	SIT does **not** match — no false positive on look-alikes.
B2	Scripted corpus — negatives	Run `CLSSIT-Test.ps1` against the negatives folder.	Zero (or below-threshold-only) matches on the negative corpus.
B3	Simulation FP baseline	Over the agreed dwell time, export simulation matches (`CLSSIT-Report.ps1 -Days <n>`), inspect them in **Content explorer**, and classify each TP/FP.	**False-positive rate** ≤ **the agreed threshold** (define with compliance — a common target is ≤10% of matches, or an absolute FP count the team can absorb).
B4	Post-tuning re-measure	After each tuning change (confidence↑, instance count↑, or a tuned SIT copy), re-run the portal/scripted test and re-simulate.	FP rate trends **down** while Test A true positives still fire.

Gate B (pass): the negative corpus is rejected and the measured false-positive rate is at/under the threshold on the simulation sample, with true positives intact.

Test C — Consistency & hand-off (the selection is usable downstream)

#	Test	Steps	Pass condition
C1	Selection register complete	Review the `SIT Selection Register`.	Every in-scope data category maps to a named built-in SIT (or is explicitly flagged as a gap for another solution).
C2	Threshold register complete	Review the `SIT Threshold Register`.	Every selected SIT has a recorded confidence level and instance count matching what was validated.
C3	Tuned copy traceable (if used)	If any `<Built-in> – Tuned Copy – Classification` was created, review its documented pattern/keyword/confidence changes and corpus test result.	The copy is documented, tested, and the original built-in SIT is confirmed **unmodified**.
C4	Downstream reference	Point a downstream control (e.g. a DLP rule, still in simulation) at the validated SIT at the registered settings.	The downstream policy resolves the SIT and applies the registered confidence/instance count without error.

Gate C (pass): both registers are complete and consistent, any tuned copy is traceable, and a downstream control consumes the validated selection cleanly.

Safety & rollback validation

#	Test	Pass condition
S1	No enforcement introduced	The `SIT Measure – … – Simulation` policy is confirmed in a **Test*** mode; no user is ever blocked by this solution's artifacts.
S2	Built-in SIT unmodified	The original built-in SITs are unchanged (only editable **copies** were tuned) — confirm in the SIT library.

#	Test	Pass condition
S3	Scripts are safe-by-default	`CLSSIT-BuildSimPolicy.ps1` and `CLSSIT-Set-Scope.ps1` run **without** `-Commit` make no change (`-WhatIf` output only); Discovery/Test/Report scripts are read-only.
S4	Clean teardown	Deleting a `SIT Measure – … – Simulation` policy removes the measurement scaffolding with no residual effect.

Evidence to capture (for the deliverable record)

• The built-in SIT inventory CSV (Phase 0) and the completed `SIT Selection Register` + `SIT Threshold Register`.

• Portal Test + scripted corpus results showing positives detected and negatives rejected (Gate A/B).

• Simulation match export + TP/FP classification and the computed **false-positive rate** before/after tuning (Gate B).

• Documentation of any tuned SIT copy and confirmation the built-in original is unmodified (S2/C3).

• Confirmation that no enforcement was introduced (S1) and the measurement policy tore down cleanly (S4).

> ***Determination boundary.*** *These tests prove the **detector is accurate**. Whether a given category's SIT selection *legally satisfies* a regulation, and whether its false-positive threshold is *acceptable*, are compliance/privacy/legal sign-offs, captured alongside the evidence above.*

6. Troubleshooting — Sensitive Information Types Implementation (Microsoft Purview)

Common issues, grouped by phase. Each has a **symptom**, the **likely cause**, and a **resolution**. This solution introduces **no enforcement**: all measurement is in the portal **Test** experience or a DLP policy in **simulation**. `CLSSIT-Discovery.ps1`, `CLSSIT-Test.ps1`, and `CLSSIT-Report.ps1` are read-only; `CLSSIT-BuildSimPolicy.ps1` and `CLSSIT-Set-Scope.ps1` are `-WhatIf` by default and create/scope policies in **simulation only** (they refuse to modify an enforcing policy). A SIT that legitimately doesn't fit a category is a **gap for another solution**, not a failure here.

Connectivity & cmdlet availability

Symptom: `Connect-IPPSSession` unavailable or `Test-DataClassification` not found. **Likely cause:** `ExchangeOnlineManagement` missing/outdated, or `Test-DataClassification` isn't present

in your tenant (it is licensing-/rollout-dependent). **Resolution:** `Install-Module ExchangeOnlineManagement -Scope CurrentUser` (or `Update-Module`). If `Test-DataClassification` is absent, `CLSSIT-Test.ps1` degrades to guidance — use the portal SIT **Test** experience instead. Verify your licensing/tenant; do not assume the cmdlet exists.

Discovery & selection (Phase 0–1)

Symptom: a built-in SIT you expected isn't in `CLSSIT-Discovery.ps1` output. **Likely cause:** the SIT is region-/plan-specific, or a role gap hid it. **Resolution:** confirm the reader role and re-run; check the portal **Data classification** → **Classifiers** → **Sensitive info types**. If it genuinely doesn't exist, record the category as a **gap** for the custom-SIT / EDM / trainable-classifier solutions — never invent a built-in SIT.

Symptom: two SITs both seem to fit a category. **Resolution:** prefer the most specific built-in SIT (a country-specific national-ID SIT over a generic number pattern); record the chosen one and the rejected alternatives in the `SIT Selection Register`.

Testing against the corpus (Phase 2)

Symptom: a positive sample only matches at Medium/Low, not High. **Likely cause:** the sample lacks the supporting keywords or corroborating evidence the SIT's High confidence requires. **Resolution:** this is a real tuning signal — either accept a lower confidence for that use, or (Phase 4) raise corroboration via a tuned copy. Record it; don't force it.

Symptom: a look-alike negative matches (false positive) in testing. **Likely cause:** the built-in pattern is broad and your look-alike resembles it. **Resolution:** capture it for Phase 4 tuning (raise confidence/instance count first; copy the SIT and add a required nearby keyword only if the dials aren't enough). Use clearly-fake data in the corpus (Requirement 2.4).

Building the simulation policy (Phase 3)

Symptom: `CLSSIT-BuildSimPolicy.ps1` made no change. **Likely cause:** `-Commit` was omitted (default is `-WhatIf`). **Resolution:** re-run with `-Commit`; the policy is created in **simulation** (`Test*` mode) only.

Symptom: the tool refuses with "policy is currently ENFORCING". **Likely cause:** a policy of that name is in `Enable` mode. **Resolution:** this is the safety guard — never let this solution flip a policy to enforce. Use a distinct `SIT Measure – <Category> – Simulation` name, or adjust the enforcing policy separately under change control.

Symptom: simulation shows no matches after creation. **Likely cause:** it hasn't dwelt long enough, unified audit-log ingestion is off, or the scope is too narrow. **Resolution:** let it dwell for the agreed time; verify ingestion (`Get-AdminAuditLogConfig | Select UnifiedAuditLogIngestionEnabled`); widen the scope if needed.

Scoping to a pilot ring (CLSSIT-Set-Scope.ps1)

Symptom: `-IncludeGroup` errors or scopes nothing. **Likely cause:** the group isn't mail-enabled, or the rule name doesn't match. **Resolution:** `-IncludeGroup` must be a **mail-enabled** group; the rule defaults to `"<PolicyName> - Rule"` — pass `-RuleName` if yours differs. Preview with `-WhatIf` first.

Symptom: SharePoint/OneDrive scope didn't narrow as expected. **Likely cause:** `-AddSharePointLocation` **adds** an included site but the broad `All` location may still apply. **Resolution:** to limit to only specific sites, also remove the broad `All` in the portal or set the location explicitly, then verify the resulting scope. DLP location scoping differs per workload.

Measuring & tuning (Phase 4)

Symptom: `CLSSIT-Report.ps1` returns no events. **Likely cause:** audit ingestion off, insufficient dwell, or nothing matched. **Resolution:** enable ingestion, allow more dwell time, and re-run. No matches is a legitimate result, not an error.

Symptom: false-positive rate won't come down. **Likely cause:** stacking multiple changes at once, or the built-in pattern is inherently broad for your data. **Resolution:** change **one dial at a time** (confidence, then instance count), re-measure; only then create a tuned copy of the built-in SIT and add a required keyword. The original built-in SIT is never modified.

Validation & handoff (Phase 5)

Symptom: downstream DLP/labeling teams get different detection results. **Likely cause:** they referenced the SIT at a different confidence/instance count than the registered values. **Resolution:** the `SIT Selection Register` and `SIT Threshold Register` are authoritative — downstream controls must consume the SIT at the **registered** confidence and instance count. Decommission the measurement policies once validated (they were scaffolding, never enforcement).

What this solution does NOT decide

Which data categories are in scope, *which* SITs legally **satisfy** a regulation, and the *acceptable* false-positive threshold are **compliance / privacy / legal determinations** — this solution delivers an accurate, tested, tuned detection layer and the evidence; adequacy and applicability belong to those functions.

7. Best Practices — Sensitive Information Types Implementation (Microsoft Purview)

Field-tested practices for selecting, testing, and tuning **built-in** sensitive information types (SITs) so the detection layer the whole Purview estate depends on stays accurate and low-noise. These complement the Implementation and Testing guides.

Reuse built-in before you build anything

- The built-in SIT library is large and **tested by Microsoft** — start there for every category. A well-

chosen built-in SIT beats a hand-rolled custom one for the common data patterns (payment cards, national IDs, IBANs, health identifiers).

- **Pick the most specific SIT.** A country-specific national-ID SIT is more accurate than a generic number

pattern. Prefer specificity; it cuts false positives at selection time, before any tuning.

• Only flag a **genuine gap** (an internal identifier, or a conceptual category like source code) for the

custom-SIT / EDM / trainable-classifier solutions — don't force a built-in SIT to do a job it wasn't built for.

Tune the two dials before touching the pattern

• **Confidence level and instance count are your first and cheapest levers.** Start at **High confidence**

and a **realistic instance count** — a single credit-card match is noise; ten in one item is a likely leak. These two settings, changed per policy at no cost, resolve most false positives.

• **You cannot edit a built-in SIT's pattern** — and you usually shouldn't need to. Reach for an **editable

copy **only when the two dials genuinely can't separate your true positives from your look-alikes (e.g. your internal reference numbers share the exact format). Tune the** copy**; never assume the built-in original changed.

• **Set the dials once, in the** `SIT Threshold Register`, and reuse them everywhere the SIT is consumed —

DLP, auto-labeling, retention — so detection is consistent across the estate.

Test with fake data, both ways

• Every SIT test uses **clearly-fake sample data** — documented test values for positives, synthetic look-

alikes for negatives. **Never** use production sensitive data in a corpus.

• **Test the negatives as hard as the positives.** A SIT that catches every real card number but also fires

on internal order IDs is a false-positive machine. The look-alike corpus is what proves the SIT is safe to turn on downstream.

• Use the portal **Test** experience for a quick per-SIT check and the scripted corpus run for scale — score

the whole corpus before and after every tuning change.

Measure against reality, in simulation

• The corpus proves a SIT in the lab; **a DLP policy in SIMULATION proves it against production content**,

where the real false-positive drivers live. Always measure in simulation — zero user impact — before calling a SIT "tuned".

• Give the simulation enough **dwell time** to capture representative traffic; a one-hour sample isn't a

false-positive rate.

• **Inspect the actual matches in Content Explorer** (with the content-viewer role) — a match count without

seeing the items can't be classified TP vs FP. Treat the content-viewer grant as sensitive and time-box it to the tuning exercise.

Keep the selection consistent and documented

• Maintain the `SIT Selection Register` (category → SIT → why) and `SIT Threshold Register`

(confidence + instance count) as the single source of truth. Every downstream policy references these — drift here means inconsistent detection everywhere.

• **Version any tuned SIT copy** (pattern/keyword/proximity/confidence, owner, date) so changes are

traceable and reversible.

• Re-validate periodically — the built-in library grows and improves, and a newer or more specific SIT may

replace a copy you once needed.

Stay inside the boundary

• A SIT detects a **pattern, not meaning** — verify it against your real look-alike data rather than trusting

the name. Conceptual categories need **trainable classifiers**, not a built-in SIT.

• **Verify licensing/tenant** before relying on Content/Activity Explorer, auto-labeling, or any advanced

measurement surface — plans and portal paths evolve; don't assume a feature or an exact library count is present.

• **Which SITs legally satisfy a regulation, and the acceptable false-positive threshold per category, are

compliance/legal determinations** — you provide the accurate detector and the evidence; they own the decision.

8. Deliverables Summary — Sensitive Information Types Implementation (Microsoft Purview)

What you receive in this solution, and what "done" looks like.

Documentation

Document	What it gives you
Solution Overview	The built-in SIT model — the library, confidence levels and instance count as the two tuning dials, where SITs are consumed across Purview, and the select→test→tune journey; naming convention.

Document	What it gives you
Requirements Guide	Solution-specific prerequisites (licensing to view/test/measure SITs, SIT roles **including Content Explorer viewer**, classification intent, a fake positive+negative test corpus, DLP for simulation, an FP threshold + pilot ring) each with verify/implement steps, on top of the six category prerequisites.
Implementation Guide	The exact ordered phases: inventory the built-in library → select per category → test against the corpus → measure in **simulation** → tune the dials (and, only if needed, an editable SIT copy) → validate and hand off; each phase named, gated, and with a rollback.
Testing & Validation	Pass/fail gates for the three headline outcomes — true positives detected (A), negatives rejected + low false-positive rate (B), selection consistent and handed off (C) — plus safety/rollback checks, all with fake test data.
Troubleshooting Guide	The common failure modes (SIT not matching, look-alike false positives, can't see matched items, Content Explorer empty) and how to resolve them.
Best Practices	Reuse-built-in-first, tune-the-dials-before-the-pattern, test-negatives-too, measure-in-simulation, and keep-the-registers-consistent.
Executive Summary	One-page business framing for approvers.

PowerShell tools (5) — Security & Compliance PowerShell / Graph

All tools are **read-only or `-WhatIf` by default**; the one tool that writes creates a policy **in simulation mode only — never enforce** — and requires an explicit `-Commit`. No credentials are stored in code. They connect via `Connect-IPPSSession` and/or `Connect-MgGraph`.

#	Script	Role	Default safety
1	**CLSSIT-Discovery.ps1**	Built-in SIT inventory: lists the SITs available in the tenant, the confidence levels each publishes, and which existing DLP/auto-labeling policies already reference SITs.	Read-only

#	Script	Role	Default safety
2	**CLSSIT-Test.ps1**	SIT accuracy tester: runs corpus files (positive + negative) through a selected SIT and reports match/no-match and confidence per file, for corpus scoring.	Read-only
3	**CLSSIT-BuildSimPolicy.ps1**	Measurement policy builder: creates a DLP policy referencing the selected SIT(s) at a chosen confidence/instance count **in simulation mode**, for real-traffic false-positive measurement.	`-WhatIf` (writes need `-Commit`; simulation-only, never enforce)
4	**CLSSIT-Report.ps1**	Match / false-positive report: exports simulation match activity for TP-vs-FP classification and false-positive-rate calculation.	Read-only
5	**CLSSIT-Set-Scope.ps1**	Scope helper: reports and (with `-Commit`) adjusts the measurement policy's included/excluded users/groups (e.g. the pilot ring).	`-WhatIf` (writes need `-Commit`)

Definition of done

• The built-in SIT library inventoried and mapped: every in-scope data category → a named built-in SIT in

the `SIT Selection Register`, or explicitly flagged as a gap for the custom/EDM/trainable solutions.

• Each selected SIT **tested** against a fake positive+negative corpus — positives detected at the target confidence, negatives rejected.

• Each selected SIT **measured** against real traffic in a **simulation** DLP policy, with matches inspected in Content Explorer and classified TP/FP.

• False-positive rate **tuned** to at/under the agreed threshold via confidence level, instance count, and (only where necessary) an editable SIT copy — with the original built-in SIT unmodified.

- The `SIT Selection Register` and `SIT Threshold Register` finalized and handed to the downstream controls;

measurement policies decommissioned; evidence captured.

> ***Boundary:*** *the solution delivers the accurate, validated* ***detector*** *and the evidence. The* ***data categories in scope, the SITs that legally satisfy a regulation, and the acceptable false-positive threshold per category*** *are compliance/privacy/legal determinations, captured alongside the evidence. Building custom SITs, Exact Data Match, and trainable classifiers for gaps is delivered by the adjacent solutions.*

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

CLSSIT-BuildSimPolicy.ps1

Purpose: Creates a DLP compliance policy (and a matching content rule) that references your selected built-in sensitive information type(s) at a chosen confidence level and instance count, in SIMULATION mode, so you can MEASURE how the SIT behaves against real production content with zero user impact. This is measurement scaffolding, not enforcement. Safe by default: with -WhatIf it only reports the intended change; it applies nothing unless -Commit is passed, and it NEVER creates or leaves a policy in enforce (Enable) mode — simulation only. HOW TO RUN Preview a simulation measurement policy for the built-in Credit Card SIT: powershell -File .\CLSSIT-BuildSimPolicy.ps1 -PolicyName 'SIT Measure - Payment Card - Simulation' `-SensitiveTypes 'Credit Card Number' -MinCount 10 -Confidence High -WhatIf` Actually create it (still simulation mode): `powershell -File .\CLSSIT-BuildSimPolicy.ps1 -PolicyName 'SIT Measure - Payment Card - Simulation'` -SensitiveTypes 'Credit Card Number' -MinCount 10 -Confidence High -Commit Requires Security & Compliance PowerShell (Connect-IPPSSession). EXPECTED RESULTS -WhatIf: prints the New-DlpCompliancePolicy / New-DlpComplianceRule calls it WOULD run, Mode=Test*. -Commit: creates the policy in simulation + the rule, then prints the resulting Mode (a Test* value = PASS). Let it dwell, then measure with CLSSIT-Report.ps1. Verify with CLSSIT-Discovery.ps1. .NOTES Purview — Sensitive Information Types Implementation. Writes gated behind -Commit; simulation-only (never Enable). No credentials in code. > [CmdletBinding()] param([Parameter(Mandatory)][string]$PolicyName, [string[]]$SensitiveTypes = @('Credit Card Number'), [int]$MinCount = 10, [ValidateSet('Low','Medium','High')][string]$Confidence = 'High', Simulation flavour: with or without showing policy tips to users during the test. [ValidateSet('TestWithNotifications','TestWithoutNotifications')][string]$SimulationMode = 'TestWithoutNotifications', Locations to measure across. Broad by default so the sample is representative. [ValidateSet('Exchange','SharePoint','OneDrive','Teams')] [string[]]$Locations = @('Exchange','SharePoint','OneDrive','Teams'), [string]$UserPrincipalName, [switch]$Commit)

How to run: Preview a simulation measurement policy for the built-in Credit Card SIT: powershell -File .\CLSSIT-BuildSimPolicy.ps1 -PolicyName 'SIT Measure - Payment Card - Simulation' `-SensitiveTypes 'Credit Card Number' -MinCount 10 -Confidence High -WhatIf` Actually create it (still simulation mode): `powershell -File .\CLSSIT-BuildSimPolicy.ps1 -PolicyName 'SIT Measure - Payment Card - Simulation'` -SensitiveTypes 'Credit Card Number' -MinCount 10 -Confidence High -Commit Requires Security & Compliance PowerShell (Connect-IPPSSession). EXPECTED RESULTS -WhatIf: prints the New-DlpCompliancePolicy / New-DlpComplianceRule calls it WOULD run, Mode=Test*. -Commit: creates the policy in simulation + the rule, then prints the resulting Mode (a Test* value = PASS). Let it dwell, then measure with CLSSIT-Report.ps1. Verify with CLSSIT-Discovery.ps1. .NOTES Purview — Sensitive Information Types Implementation. Writes gated behind -Commit; simulation-only (never Enable). No credentials in code. > [CmdletBinding()] param([Parameter(Mandatory)][string]$PolicyName, [string[]]$SensitiveTypes = @('Credit Card Number'), [int]$MinCount = 10, [ValidateSet('Low','Medium','High')][string]$Confidence = 'High', Simulation flavour: with or without showing policy tips to users during the test. [ValidateSet('TestWithNotifications','TestWithoutNotifications')][string]$SimulationMode = 'TestWithoutNotifications', Locations to measure across. Broad by default so the sample is representative.

[ValidateSet('Exchange','SharePoint','OneDrive','Teams')] [string[]]$Locations = @('Exchange','SharePoint','OneDrive','Teams'), [string]$UserPrincipalName, [switch]$Commit)

CLSSIT-Discovery.ps1

Purpose: Builds the SIT baseline: lists the sensitive information types available in the tenant (built-in vs custom), the confidence levels each publishes, and cross-references which existing DLP compliance policies (and auto-labeling policies, where available) already reference SITs. Read-only — makes no changes. Use it before selecting built-in SITs so you build on what exists. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CLSSIT-Discovery.ps1 -CsvPath .\CLSSIT-SIT-Inventory.csv Requires Security & Compliance PowerShell (Connect-IPPSSession via ExchangeOnlineManagement). EXPECTED RESULTS A console table of SITs (Name, Publisher, Built-in?, Confidence levels) and a summary of how many DLP policies reference SITs. PASS signal: the built-in SITs for your in-scope categories appear in the list — record them in the SIT Selection Register. .NOTES Purview — Sensitive Information Types Implementation. Read-only against Security & Compliance PowerShell. > [CmdletBinding()] param([string]$CsvPath, Optional case-insensitive filter, e.g. -NameLike 'credit','ssn','iban' [string[]]$NameLike, [string]$UserPrincipalName)

How to run: powershell -ExecutionPolicy Bypass -File .\CLSSIT-Discovery.ps1 -CsvPath .\CLSSIT-SIT-Inventory.csv Requires Security & Compliance PowerShell (Connect-IPPSSession via ExchangeOnlineManagement). EXPECTED RESULTS A console table of SITs (Name, Publisher, Built-in?, Confidence levels) and a summary of how many DLP policies reference SITs. PASS signal: the built-in SITs for your in-scope categories appear in the list — record them in the SIT Selection Register. .NOTES Purview — Sensitive Information Types Implementation. Read-only against Security & Compliance PowerShell. > [CmdletBinding()] param([string]$CsvPath, Optional case-insensitive filter, e.g. -NameLike 'credit','ssn','iban' [string[]]$NameLike, [string]$UserPrincipalName)

CLSSIT-Report.ps1

Purpose: Reads what the SIT measurement policy WOULD have matched (simulation mode) from the unified audit log over the last N days and exports one row per match event — policy, rule, workload, and the sensitive-information-type hit — so you can classify each as true positive or false positive and find the false-positive drivers during tuning (Phase 4). Read-only: it reads audit events only and changes nothing. For the actual matched CONTENT, cross-reference Purview Content Explorer (needs the Content Explorer viewer role) — this tool points you there; it does not read item bodies. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CLSSIT-Report.ps1 -Days 14 -CsvPath .\CLSSIT-Matches.csv powershell -ExecutionPolicy Bypass -File .\CLSSIT-Report.ps1 -Days 14 -PolicyName 'SIT Measure - Payment Card - Simulation' Requires Security & Compliance PowerShell (Connect-IPPSSession) and permission to run Search-UnifiedAuditLog. Unified audit-log ingestion must be enabled. PowerShell 5.1+ or 7+. EXPECTED RESULTS A per-match table + optional CSV of simulation matches (policy/rule/workload/SIT). If audit ingestion is off or no matches yet, that is reported (not an error) — let the policy dwell longer. .NOTES Purview — Sensitive Information Types Implementation. Read-only. Whether a match is a true/false positive and the acceptable FP threshold are your tuning decisions; SIT regulatory sufficiency is a compliance/legal determination. > [CmdletBinding()] param([int]$Days = 14, [string]$PolicyName, # optional: filter to one measurement policy [string]$CsvPath, [string]$UserPrincipalName)

How to run: powershell -ExecutionPolicy Bypass -File .\CLSSIT-Report.ps1 -Days 14 -CsvPath .\CLSSIT-Matches.csv powershell -ExecutionPolicy Bypass -File .\CLSSIT-Report.ps1 -Days 14 -PolicyName 'SIT Measure - Payment Card - Simulation' Requires Security & Compliance PowerShell

(Connect-IPPSSession) and permission to run Search-UnifiedAuditLog. Unified audit-log ingestion must be enabled. PowerShell 5.1+ or 7+. EXPECTED RESULTS A per-match table + optional CSV of simulation matches (policy/rule/workload/SIT). If audit ingestion is off or no matches yet, that is reported (not an error) — let the policy dwell longer. .NOTES Purview — Sensitive Information Types Implementation. Read-only. Whether a match is a true/false positive and the acceptable FP threshold are your tuning decisions; SIT regulatory sufficiency is a compliance/legal determination. > [CmdletBinding()] param([int]$Days = 14, [string]$PolicyName, # optional: filter to one measurement policy [string]$CsvPath, [string]$UserPrincipalName)

CLSSIT-Set-Scope.ps1

Purpose: Scopes the simulation measurement policy so the sample is limited to a pilot ring rather than the whole tenant. For Exchange it restricts the rule to senders who are members of a mail-enabled group (Set-DlpComplianceRule -FromMemberOf); for SharePoint/OneDrive it can include specific site URLs. Previews the exact cmdlet under -WhatIf and changes the policy ONLY with -Commit. It never enables enforcement — the policy stays in simulation. Refuses to touch a policy that is in enforce (Enable) mode. HOW TO RUN Preview: limit the Exchange scope to a pilot group's senders powershell -ExecutionPolicy Bypass -File .\CLSSIT-Set-Scope.ps1 -PolicyName 'SIT Measure - Payment Card - Simulation' -IncludeGroup 'sit-pilot-ring@contoso.com' -WhatIf Apply powershell -ExecutionPolicy Bypass -File .\CLSSIT-Set-Scope.ps1 -PolicyName 'SIT Measure - Payment Card - Simulation' -IncludeGroup 'sit-pilot-ring@contoso.com' -Commit Optionally scope SharePoint/OneDrive to specific sites ... -IncludeSites 'https://contoso.sharepoint.com/sites/pilot' -Commit Requires Security & Compliance PowerShell (Connect-IPPSSession). PowerShell 5.1+ or 7+. EXPECTED RESULTS -WhatIf: prints the Set-DlpComplianceRule / Set-DlpCompliancePolicy it WOULD run. -Commit: applies the scope and re-reads to confirm the policy is still in a Test* (simulation) mode. .NOTES Purview — Sensitive Information Types Implementation. Safe-by-default (-WhatIf); simulation-only (never Enable). -IncludeGroup must be a mail-enabled group. DLP location scoping differs per workload — verify. > [CmdletBinding()] param([Parameter(Mandatory)][string]$PolicyName, [string]$IncludeGroup, # mail-enabled group for Exchange sender scoping [string[]]$IncludeSites, # SharePoint/OneDrive site URLs to include [string]$RuleName, # defaults to "<PolicyName> - Rule" [switch]$Commit, [string]$UserPrincipalName)

How to run: Preview: limit the Exchange scope to a pilot group's senders powershell -ExecutionPolicy Bypass -File .\CLSSIT-Set-Scope.ps1 -PolicyName 'SIT Measure - Payment Card - Simulation' -IncludeGroup 'sit-pilot-ring@contoso.com' -WhatIf Apply powershell -ExecutionPolicy Bypass -File .\CLSSIT-Set-Scope.ps1 -PolicyName 'SIT Measure - Payment Card - Simulation' -IncludeGroup 'sit-pilot-ring@contoso.com' -Commit Optionally scope SharePoint/OneDrive to specific sites ... -IncludeSites 'https://contoso.sharepoint.com/sites/pilot' -Commit Requires Security & Compliance PowerShell (Connect-IPPSSession). PowerShell 5.1+ or 7+. EXPECTED RESULTS -WhatIf: prints the Set-DlpComplianceRule / Set-DlpCompliancePolicy it WOULD run. -Commit: applies the scope and re-reads to confirm the policy is still in a Test* (simulation) mode. .NOTES Purview — Sensitive Information Types Implementation. Safe-by-default (-WhatIf); simulation-only (never Enable). -IncludeGroup must be a mail-enabled group. DLP location scoping differs per workload — verify. > [CmdletBinding()] param([Parameter(Mandatory)][string]$PolicyName, [string]$IncludeGroup, # mail-enabled group for Exchange sender scoping [string[]]$IncludeSites, # SharePoint/OneDrive site URLs to include [string]$RuleName, # defaults to "<PolicyName> - Rule" [switch]$Commit, [string]$UserPrincipalName)

CLSSIT-Test.ps1

Purpose: Scores a sensitive information type against a folder of clearly-fake corpus files (positive and negative / look-alike samples). For each file it reports whether the SIT matched and at what confidence, so you can prove positives are detected and look-alike negatives are rejected before you build any policy. Read-only — classifies text, changes nothing. Uses Test-DataClassification where present in the tenant; if the cmdlet is unavailable it prints the portal Test steps instead. HOW TO RUN powershell -File .\CLSSIT-Test.ps1 -SensitiveType 'Credit Card Number' -Path .\corpus\payment-card\ -CsvPath .\CLSSIT-TestResults.csv Requires Security & Compliance PowerShell (Connect-IPPSSession). Use ONLY fake/synthetic data. EXPECTED RESULTS A per-file table: FileName, Matched (True/False), Confidence, InstanceCount. PASS signal: files in a 'positives' path match at the target confidence; files in a 'negatives' path do not match. .NOTES Purview — Sensitive Information Types Implementation. Read-only. Never place real sensitive data in the corpus. > [CmdletBinding()] param([Parameter(Mandatory)][string]$SensitiveType, [Parameter(Mandatory)][string]$Path, [string]$CsvPath, [string]$UserPrincipalName)

How to run: powershell -File .\CLSSIT-Test.ps1 -SensitiveType 'Credit Card Number' -Path .\corpus\payment-card\ -CsvPath .\CLSSIT-TestResults.csv Requires Security & Compliance PowerShell (Connect-IPPSSession). Use ONLY fake/synthetic data. EXPECTED RESULTS A per-file table: FileName, Matched (True/False), Confidence, InstanceCount. PASS signal: files in a 'positives' path match at the target confidence; files in a 'negatives' path do not match. .NOTES Purview — Sensitive Information Types Implementation. Read-only. Never place real sensitive data in the corpus. > [CmdletBinding()] param([Parameter(Mandatory)][string]$SensitiveType, [Parameter(Mandatory)][string]$Path, [string]$CsvPath, [string]$UserPrincipalName)